

Developer Productivity Platform

Project Proposal for UCS503P
Thapar Institute of Engineering and Technology

Kunjal Syal
Roll No: 1024030337

Ayush Vaibhav
Roll No: 1024030333

Barleen Kaur
Roll No: 1024030321

August 17, 2026

1 Higher-order goal

...secondary framing

This project aims to improve developer productivity and software comprehension by reducing the time and effort required to understand unfamiliar codebases. While the underlying goal applies broadly to any engineering organization, the immediate engineering objective is to translate *faster, better-grounded code understanding* into a measurable, deployable software solution.

2 Time-to-value

...primary heuristic

- We will deliver meaningful increments quickly by prototyping the core indexing-and-search workflow on a single sample repository before generalizing to arbitrary codebases.
- We will prioritize fast feedback over completeness by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success in the first iteration is defined by whether a developer can ask a natural-language question about a real repository and get an answer grounded in an actual file/line, not by feature completeness.

3 Problem Statement

Developers spend significant time locating and understanding relevant code when working in unfamiliar or large codebases. Existing tools fall short in specific ways:

- **Text-only search:** tools like grep or IDE symbol search match literal strings or symbol names, not intent – a query like “where is rate limiting enforced” returns nothing unless that exact phrase appears in code.
- **Fragmented context:** the reasoning behind code decisions often lives in pull request discussions or issue threads, and design documentation lives separately in tools like Notion – neither is searchable alongside the code itself.
- **No structural understanding:** flat text search has no notion of module boundaries, dependencies, or call relationships, so answers lack the surrounding architectural context a developer actually needs.

- **Poor traceability:** answers to “why is this code written this way” rarely link back to the exact file, line, or discussion that explains it, forcing manual digging.

Why this matters now (contextual relevance): as codebases and teams grow, onboarding time and cross-team code comprehension become a measurable drag on delivery velocity – a problem that scales with repository size and team distribution.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a **web-based** codebase intelligence platform with the following components:

- **Codebase indexing:** parses a repository using AST-based analysis to build a structured representation of the code (modules, functions, classes, call/import relationships).
- **Semantic code search:** natural-language queries locate relevant code across the indexed repository, ranked by relevance.
- **AI codebase explainer:** answers developer questions about specific functions, modules, or components by retrieving relevant code and linked documentation, with citations to the exact file and line.
- **Cross-platform context integration:** connects to GitHub (code, comments, pull request and issue history) and Notion (linked project documentation) so relevant context from both sources is available during search and explanation.
- **Architecture visualization:** automatically generates a dependency and module-relationship graph from static analysis of the codebase.
- **Pull request summarization:** converts code diffs into concise, plain-language summaries with links to related prior discussion.

4.2 Core workflow

1. User connects a GitHub repository (and optionally a linked Notion workspace); the system indexes code and documentation.
2. User issues a natural-language query or asks a question about a specific function/module.
3. System retrieves relevant code chunks and documentation via semantic search, and returns either ranked results or a synthesized answer with citations to file/line.
4. For pull requests, the system generates a plain-language diff summary linked to related issue/PR discussion.

4.3 Operational constraints for an educational setting

- Scope indexing to 1–2 languages initially (e.g., Python and JavaScript) rather than attempting universal language support.
- Avoid dependence on unproven or exotic research techniques; use established retrieval-augmented generation (RAG) patterns.
- Design for deployability on standard web hosting with a managed vector database.

5 Solution Approach

... Engineering focus

This is an engineering course project, so we focus on retrieval correctness, integration reliability, and citation grounding rather than novel machine learning research.

5.1 Semantic retrieval

... non-research

Semantic search will use established, off-the-shelf components rather than custom model training:

- AST-based chunking of code into semantically meaningful units (functions, classes) using a parser library (e.g., tree-sitter).
- An off-the-shelf embedding model to vectorize code chunks and documentation.
- Vector similarity search (cosine similarity) with a standard vector database, without claiming state-of-the-art retrieval performance.
- Every AI-generated answer must cite the specific file/line it was grounded in; answers without sufficient retrieval support are flagged as low-confidence rather than fabricated.

5.2 Web architecture

... web-based solution

A typical 3-tier web architecture:

- **Frontend:** search interface, explainer chat, architecture graph viewer.
- **Backend API:** indexing pipeline, retrieval orchestration, GitHub/Notion connector logic, LLM query handling.
- **Storage:** vector database for embeddings, relational/document database for metadata (indexed repos, PR/issue cache, user sessions).

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and test the indexing pipeline and API on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features (search, then explainer, then integrations).

6 Evaluation Criterion

... measurable and attributable

We define success using measurable metrics tied to the core retrieval-and-explanation workflow.

6.1 Primary evaluation metric

Search relevance (precision@k): for a hand-labeled set of natural-language queries against a fixed test repository, the proportion of top- k returned results judged relevant.

- Target: $\text{precision@5} \geq 0.7$ on the test query set during prototype evaluation.
- Attribution: measured by manual relevance labeling against retrieved results, logged per query.

6.2 Secondary evaluation metrics

- **Citation accuracy:** percentage of explainer answers whose cited file/line actually supports the claim made (manually verified against a sample set).
- **Index coverage:** percentage of files in a target repository successfully parsed and indexed without error.
- **Latency:** median response time for a search query and for an explainer answer.
- **User clarity:** qualitative feedback from test users (1–5 rating) on whether an explainer answer was understandable and useful.

6.3 Pilot validation plan

... high-level

- Select 2–3 real open-source repositories of varying size as test subjects.
- Recruit a small group of peer developers unfamiliar with those repositories to run sample queries and rate result relevance and explainer usefulness.
- Compare time-to-locate-relevant-code with and without the tool for a fixed set of onboarding-style questions.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in repository size and query volume by:
 - decoupling indexing (offline/batch) from query serving (online),
 - using incremental re-indexing on repository changes rather than full re-index,
 - paginating and limiting retrieval results for common queries.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed vector database,
 - containerized or platform-hosted deployment,
 - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- AST parsing library (e.g., tree-sitter) for multi-language static analysis.
- Off-the-shelf embedding models and vector databases (e.g., FAISS, Chroma) for semantic retrieval.
- GitHub REST/GraphQL API and Notion API for context connectors.
- LLM API for answer synthesis and PR diff summarization.
- Standard web framework, test framework, and CI runner.

We will use these mature components to focus on retrieval quality, integration correctness, and citation grounding rather than inventing new infrastructure.

9 Project scope and deliverables

... *iteration-friendly*

9.1 Initial deliverable

... *first iteration*

- Repository indexer for 1–2 languages using AST-based parsing.
- Semantic search over indexed code, returning ranked results.
- Basic explainer: answers a question about a function/module with a file/line citation.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job.

9.2 Subsequent deliverables

- GitHub connector: PR/issue history indexed and searchable alongside code.
- Notion connector: linked documentation indexed as additional context.
- Architecture visualization: auto-generated dependency graph.
- Pull request summarization: diff-to-plain-language summaries with linked discussion.
- (*Post-MVP, time permitting*) Git contribution analytics, automatic docstring generation, automatic test scaffolding.

9.3 Explicitly out of scope

Performance profiling and bug prediction based on historical commit data are not part of this project, as they require runtime instrumentation and trained predictive models respectively, both of which are outside the current scope and timeline.

10 Risks and mitigations

- **LLM hallucination in explanations:** mitigated by strict retrieval-grounding and requiring a citation for every claim; answers without adequate retrieval support are flagged rather than fabricated.
- **AST parsing complexity across languages:** mitigated by scoping to 1–2 languages initially before generalizing.
- **GitHub/Notion API rate limits:** mitigated by caching indexed context and incremental re-indexing instead of full re-fetch on every query.
- **Retrieval quality on code vs. prose:** code-specific embedding behavior is not yet evaluated for this project; this is treated as an explicit research task during the prototype phase, not a solved problem.
- **Evaluation measurement ambiguity:** define the labeled query set and relevance criteria before development begins, to avoid post-hoc metric shopping.

11 Summary

This project proposes a deployable AI-powered codebase intelligence platform that reduces the time and effort required to understand unfamiliar code. It meets the course criteria by prioritizing fast, iteration-friendly delivery of a working search-and-explain loop, using CI/CD to support frequent safe releases, and defining measurable, attributable evaluation criteria – particularly search relevance and citation accuracy – while remaining focused on sound retrieval-engineering practice rather than unproven research novelty.